Neural Networks

- Lecture 4 -

Applications of Convolutional Neural Networks

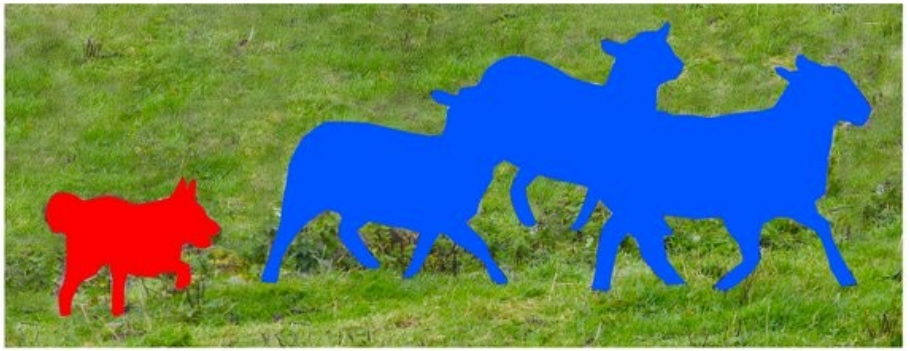
Outline

- Semantic and Instance Segmentation
- Object Detection
- CNN for Timeseries Analysis

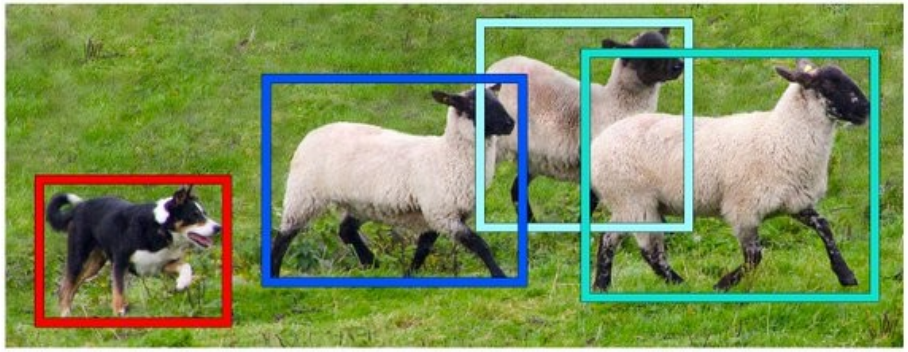
Task definitions



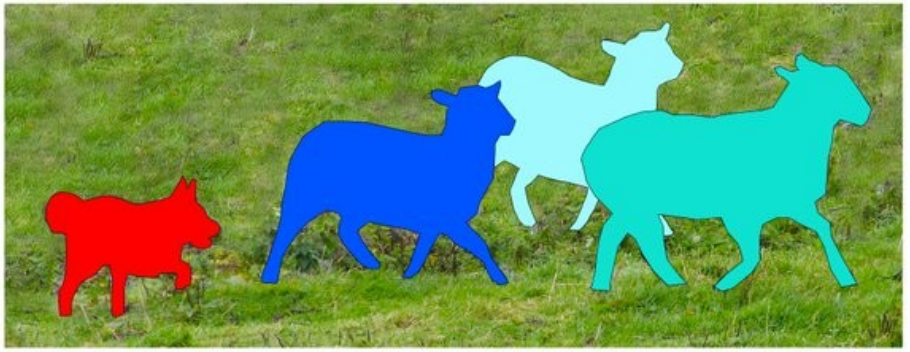
Image Recognition



Semantic Segmentation



Object Detection



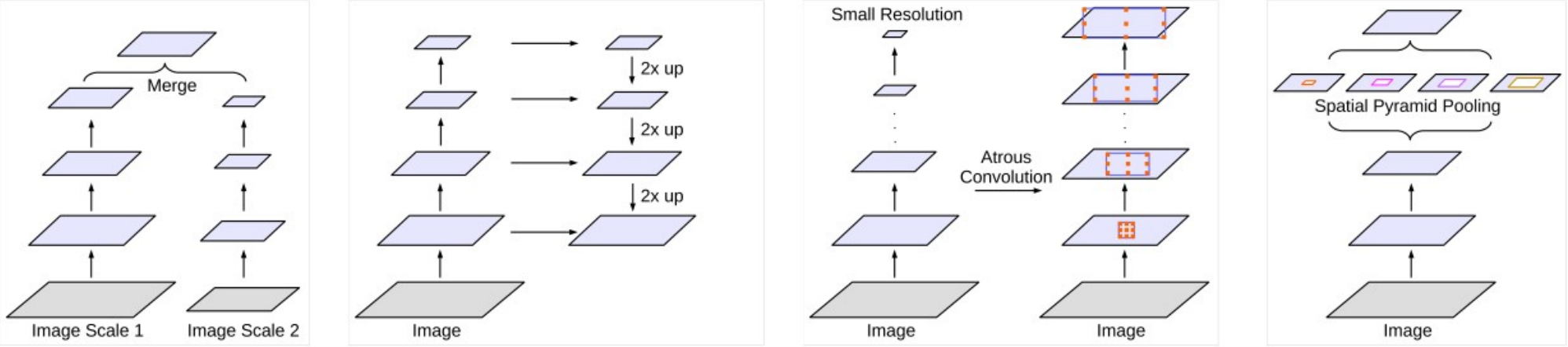
Instance Segmentation

Source: AI Pool blog: How does instance segmentation work

Semantic Segmentation

Semantic Segmentation – DeepLab-v3

- Highlights:
  - Use of **dilated convolutions (atrous convolutions)** to enable **larger effective receptive field** sizes. Helps in capturing *spatial context* information
  - Use of **Atrous Spatial Pyramid Pooling** to extract **content information** from **several scale levels** at the same time

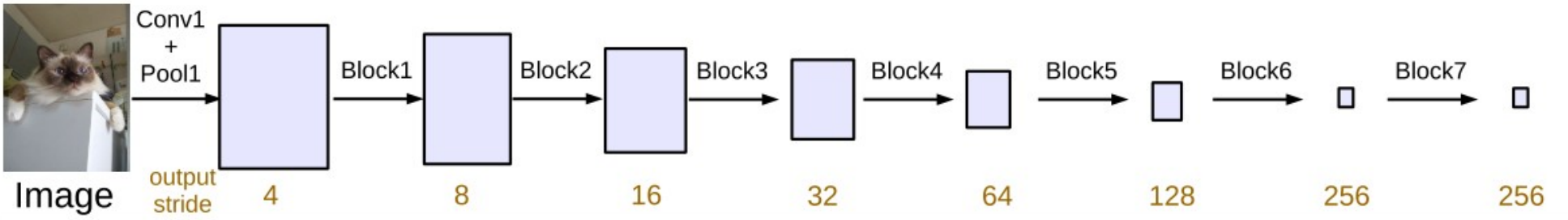


(a) Image Pyramid (b) Encoder-Decoder (c) Deeper w. Atrous Convolution (d) Spatial Pyramid Pooling

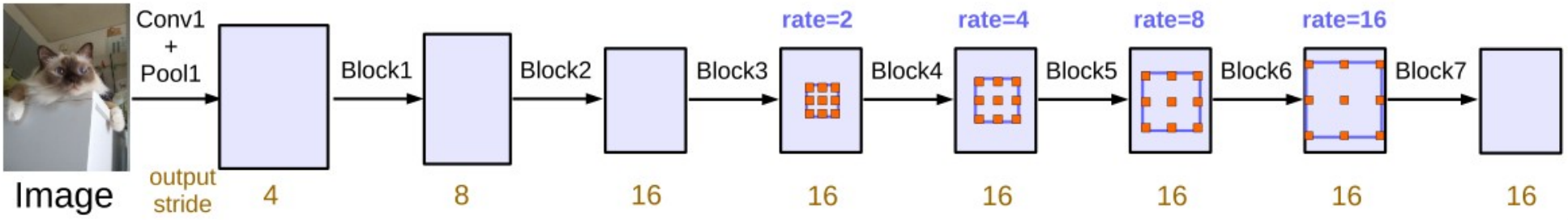
Figure 2. Alternative architectures to capture multi-scale context.

Semantic Segmentation – DeepLab-v3

- Conv Layer vs Dilated Conv Layer:
  - Use of **dilated convolutions** to extract larger *information context*
  - *Output stride* (i.e. reduction factor for image resolution) limited to 16, as larger values are *harmful* for semantic segmentation



(a) Going deeper without atrous convolution.



(b) Going deeper with atrous convolution. Atrous convolution with $rate > 1$ is applied after block3 when $output_stride = 16$.

Figure 3. Cascaded modules without and with atrous convolution.

Semantic Segmentation – DeepLab-v3

- Atrous Spatial Pyramid Pooling:
  - Uses idea that it is *effective* to resample features at different **scales** for more accurate region classification
  - Implements Spatial Pyramid Pooling using a combination of *atrous convolutions* and *global average pooling*
  - BatchNorm applied in between all diluted-conv filters

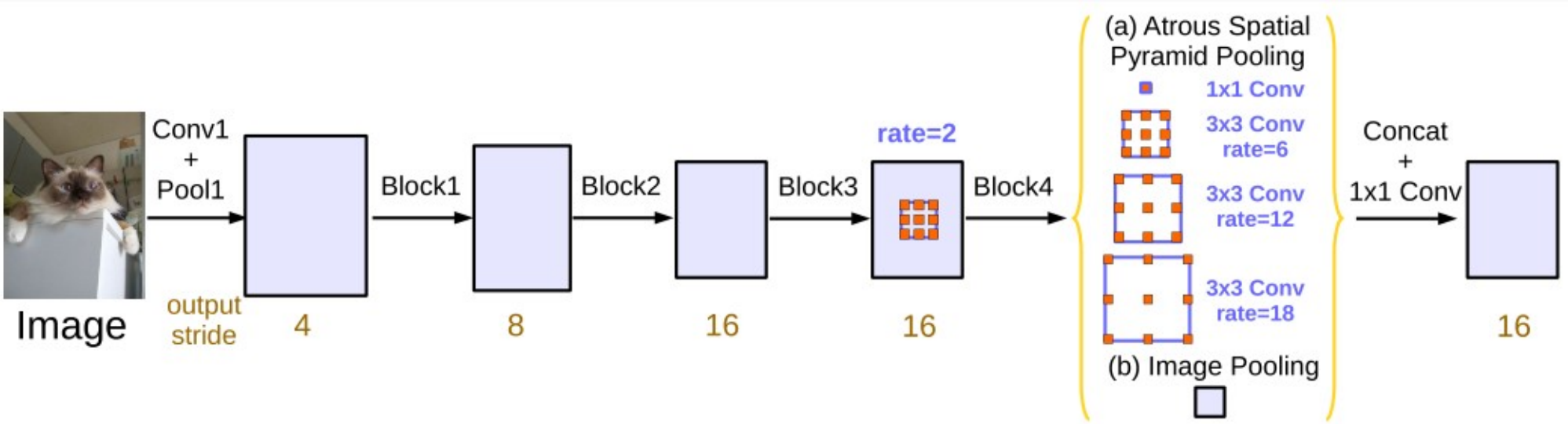


Figure 5. Parallel modules with atrous convolution (ASPP), augmented with image-level features.

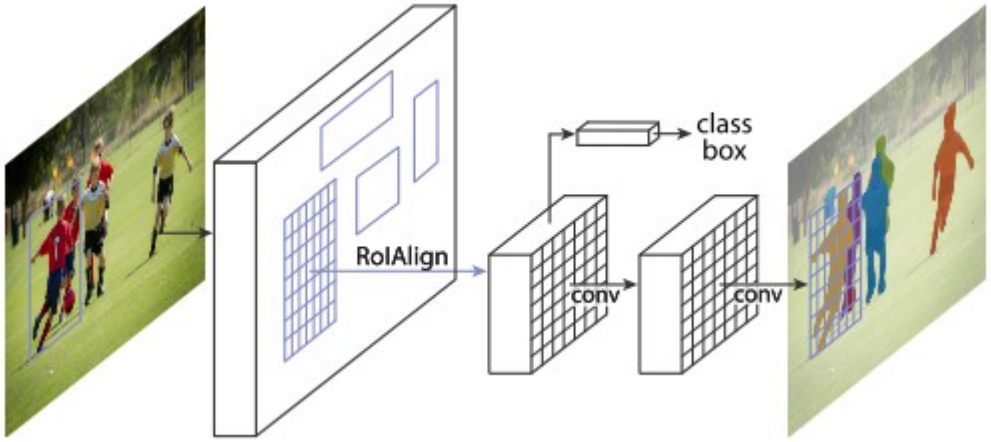
Instance Segmentation

Instance Segmentation Approaches

- Two main streams of methods in Instance Segmentation
  - **Proposal based** – propose possible **regions of interest** where an object might be found → extract features from that region → use features to classify *which pixel* is part of the object and *what class* it actually is
  - **Segmentation based** – start from *segmentation as first objective* (previously explained models) and *learn* specially designed transformations or *instance boundaries*

Instance Segmentation – the case of Mask-RCNN

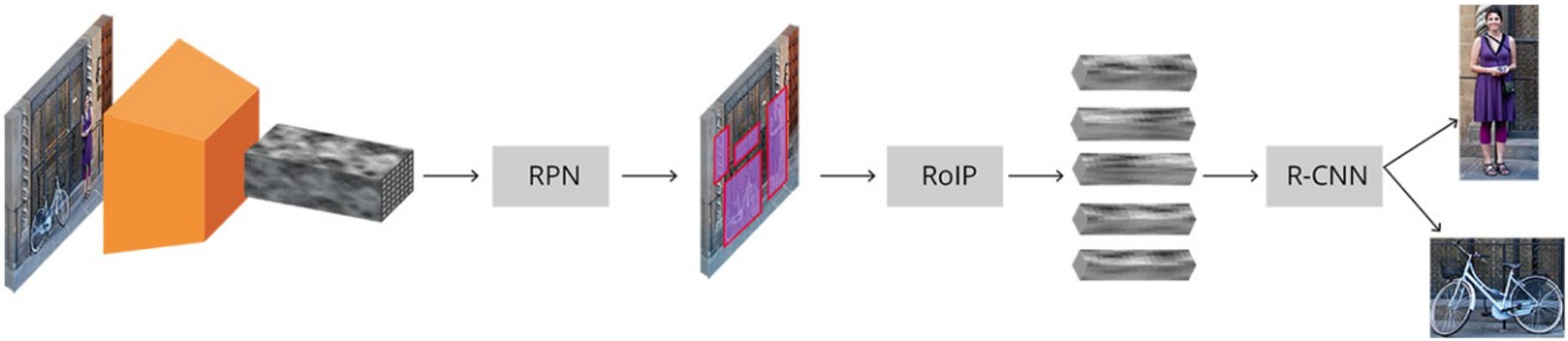
- Region of Interest (RoI) proposal based architecture
- Builds upon Faster-RCNN [Ren et al., 2015] but introduces two important aspects
  - **RoI Align Layer**
  - **binary mask prediction** (for individual instances)



The MaskR-CNN framework for instance segmentation. Source: Mask-RCNN, He et al., 2018

Instance Segmentation – A prior on Faster-RCNN

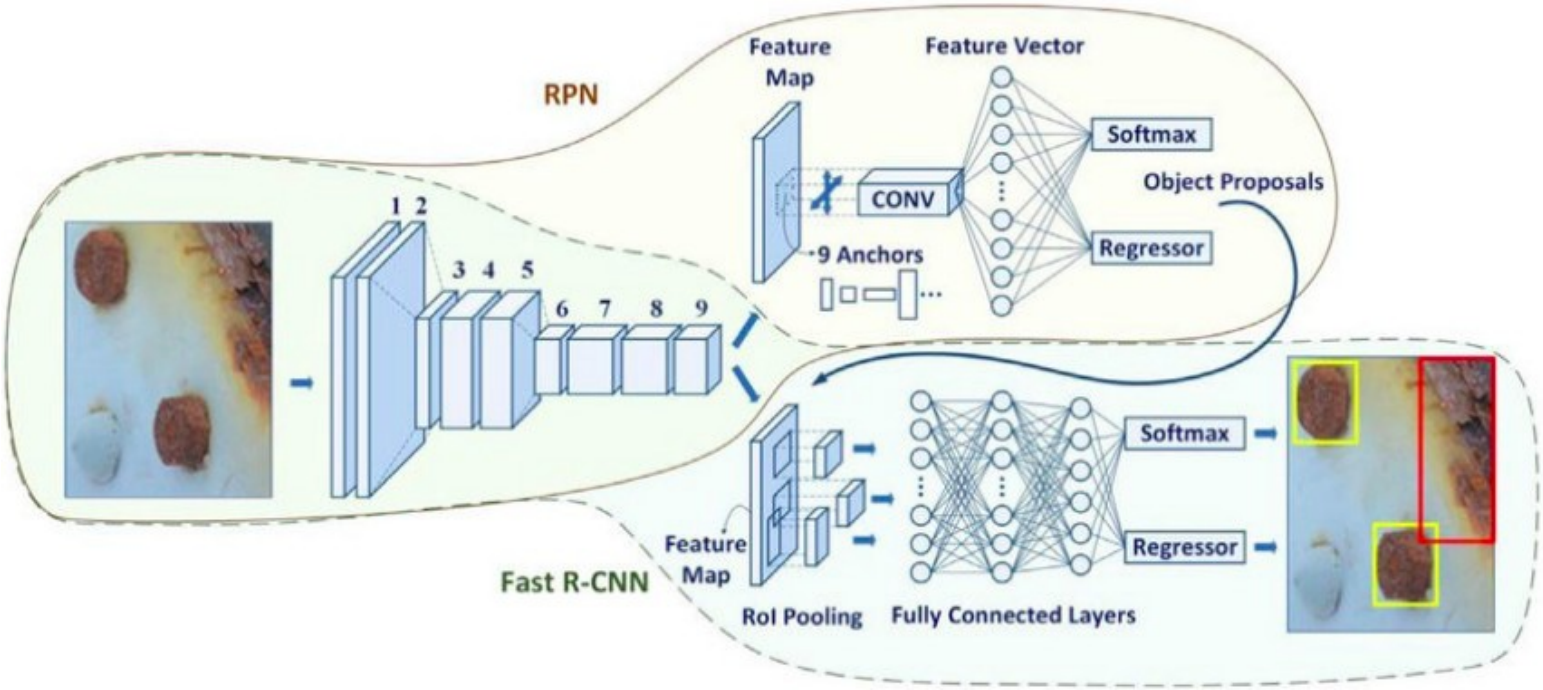
- Faster R-CNN used for *object detection*
- Fundamental concepts: **Region Proposal Network**, **Anchor Boxes**, **RoI Pooling**



The different layers of Faster R-CNN. Source: alegion.com/faster-r-cnn

Instance Segmentation – A prior on Faster-RCNN

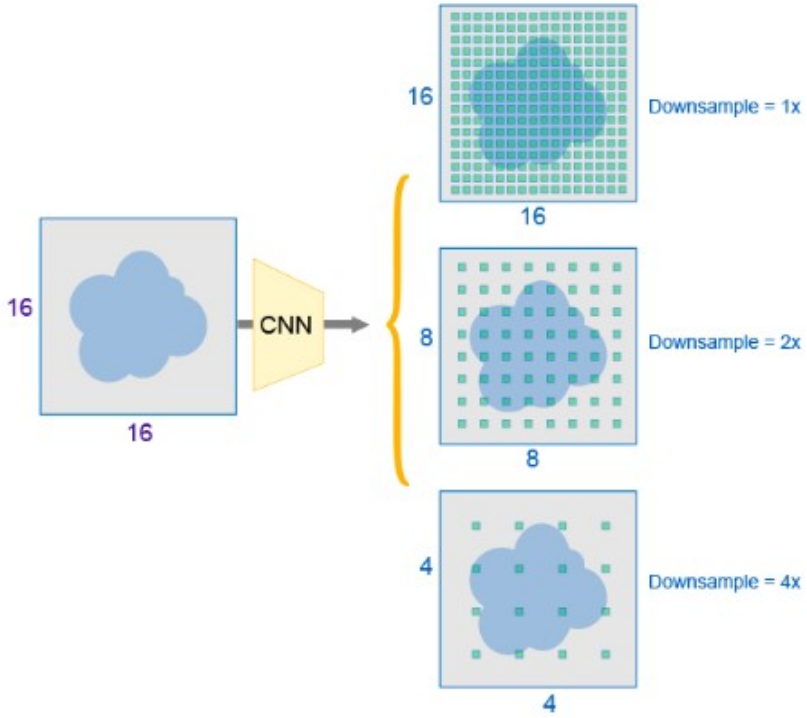
- Faster RCNN has a **backbone network (VGG-16)** which feeds the Region Proposal Network **and** the Class and BBox regressor network
- RPN proposes regions *where objects might be found*
  - Uses 9 anchor types (3 different scales, 3 different aspect ratios)
  - Predicts 2k object classification scores (whether foreground or background object) and 4k region regression coords; k – number of anchors



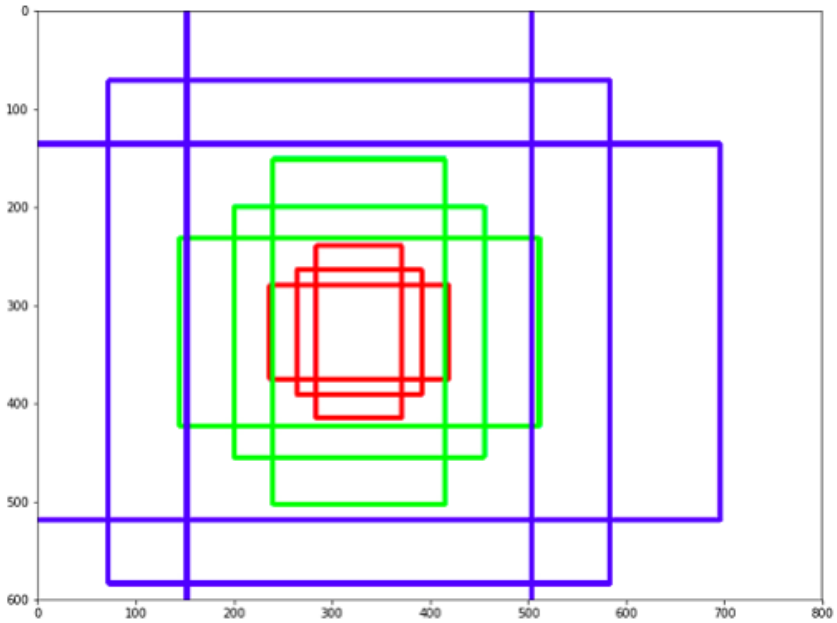
Faster-RCNN Architecture. Source: https://towardsdatascience.com/faster-rcnn-object-detection-f865e5ed7fc4

Instance Segmentation – A prior on Faster-RCNN

- Anchor Boxes
  - Take the final feature map from backbone network as input
  - Their sizes are relative to *input image*



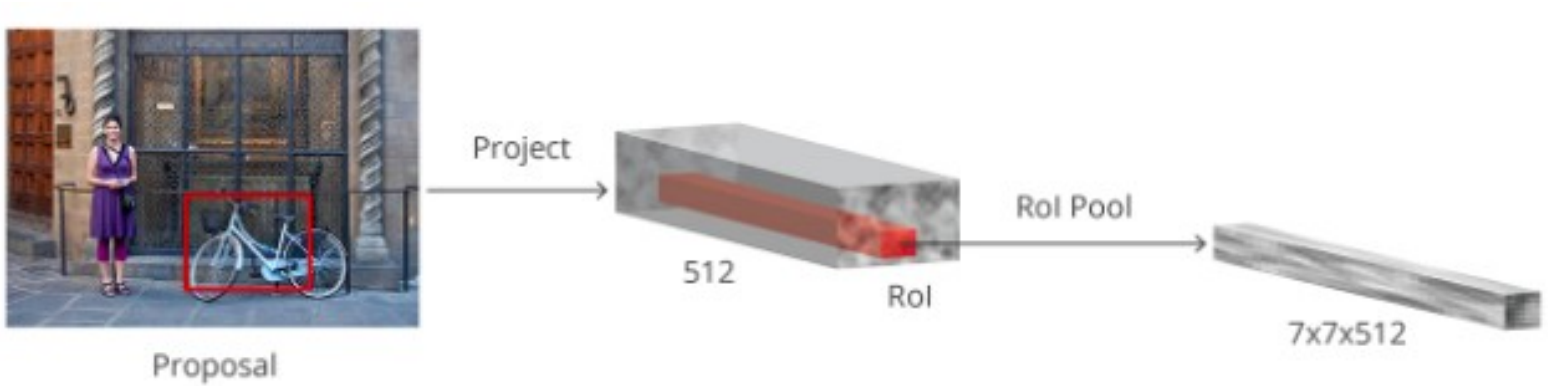
Downsampling ratios of CNN feature maps. Source: Mathworks, anchor boxes for object detection



Anchor boxes: 3 scales, 3 aspect ratios. Source: Medium @smallfishbigsea

Instance Segmentation – A prior on Faster-RCNN

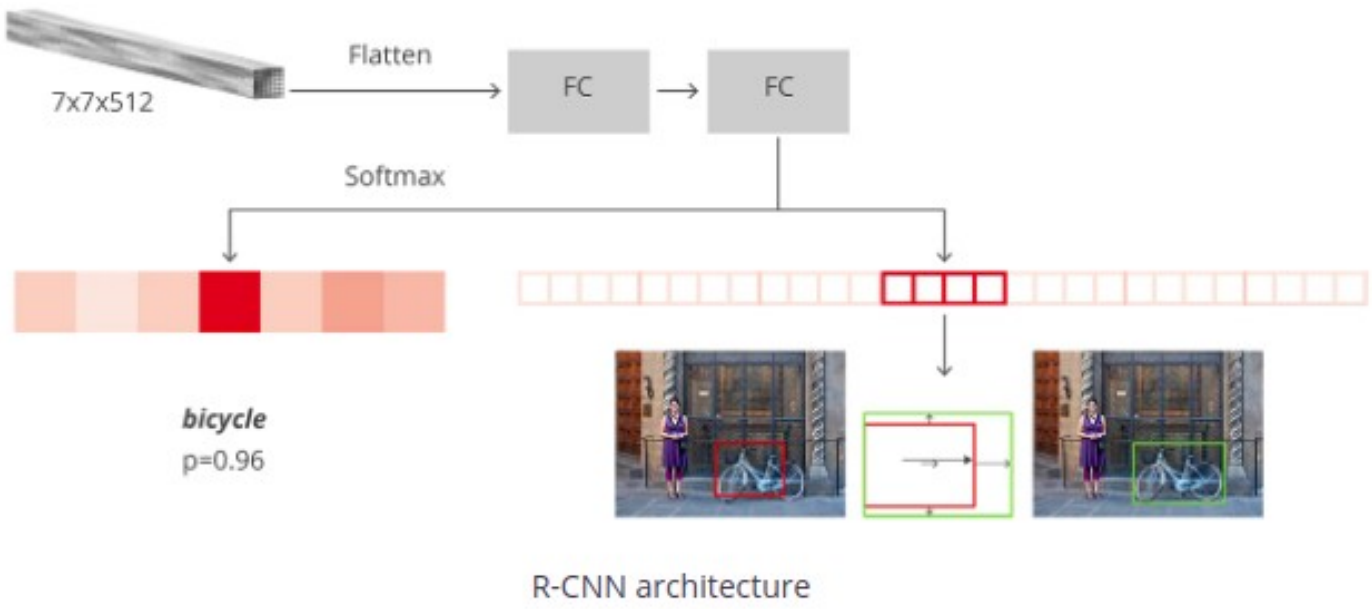
- RoI Pooling
  - For collected Region Proposals, reuse existing convolutional feature map to extract features
  - Crop the convolutional feature map using each proposal and then resize each crop to a fixed sized (e.g. 14 x 14 x convdepth) using interpolation (usually bilinear)



Region of Interest Pooling. Source: TryoLabs

Instance Segmentation – A prior on Faster-RCNN

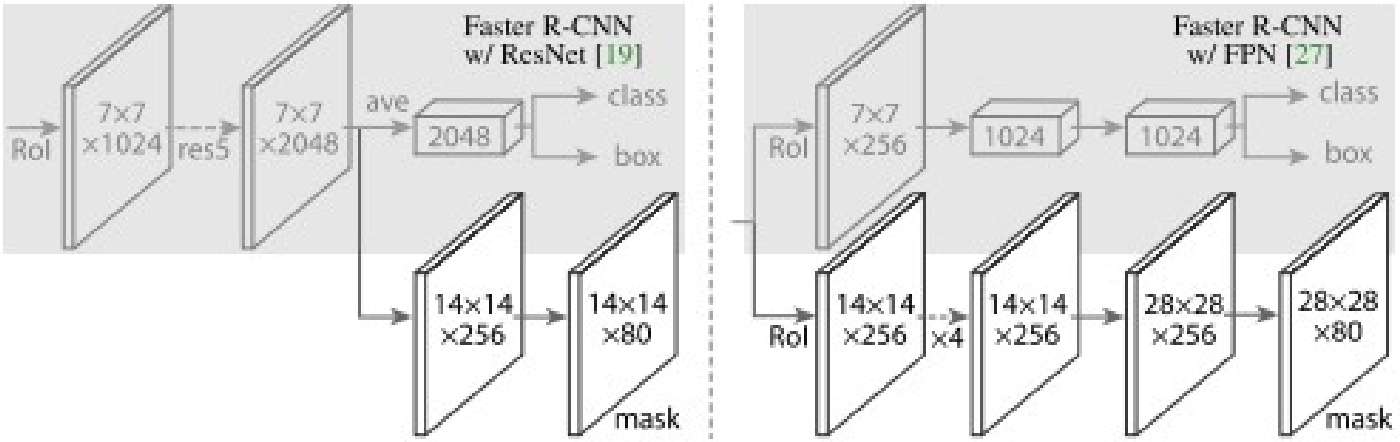
- Final Object Class Assignment and BBox fine-tuning
  - Use RoI Pooling features
  - Classify proposals into one of the classes, plus a background class
  - Better adjust BBox for proposal given the predicted class



R-CNN architecture – final object classifications and BBox regression. Source: TryoLabs

Instance Segmentation – Back to Mask-RCNN

- Mask-RCNN changes
  - Uses different backbones: ResNext-101, FPN (Feature Pyramid Network)
  - Adds a fully convolutional Mask Prediction branch



Left/Right panels show the heads for the ResNet C4 and FPN backbones, to which a mask branch is added. Numbers denote spatial resolution and channels. Arrows denote either conv, deconv, or fc layers. All convs are 3×3, except the output conv which is 1×1, deconvs are 2×2 with stride 2, and we use ReLU in hidden layers. Left: 'res5' denotes ResNet's fifth stage, which for simplicity we altered so that the first conv oper- ates on a 7×7 RoI with stride 1 (instead of 14×14 / stride 2 as in [19]).
Right: '×4' denotes a stack of four consecutive convs.

Source: Mask R-CNN, He et al., 2018

Instance Segmentation – Back to Mask-RCNN

- Mask-RCNN changes
  - Replaces RoI Pooling with a RoI Align Layer
    - Avoids quantization of RoI coordinates or spatial bins to feature map grid

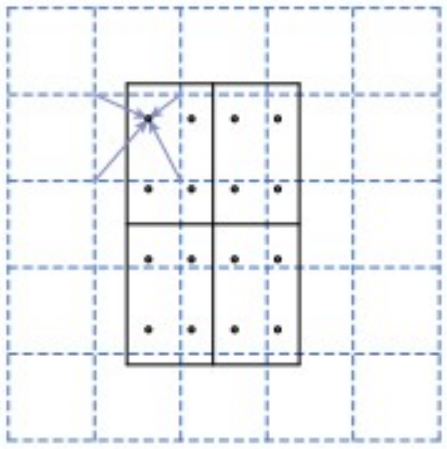


Figure 3. **RoIAlign:** The dashed grid represents a feature map, the solid lines an RoI (with 2×2 bins in this example), and the dots the 4 sampling points in each bin. RoIAlign computes the value of each sampling point by bilinear interpolation from the nearby grid points on the feature map. No quantization is performed on any coordinates involved in the RoI, its bins, or the sampling points.

RoI Align Layer in action. Source: Mask R-CNN, He et al., 2018

Instance Segmentation – Back to Mask-RCNN

- Mask-RCNN changes
  - Replaces RoI Pooling with a RoI Align Layer
    - Avoids quantization of RoI coordinates or spatial bins to feature map grid

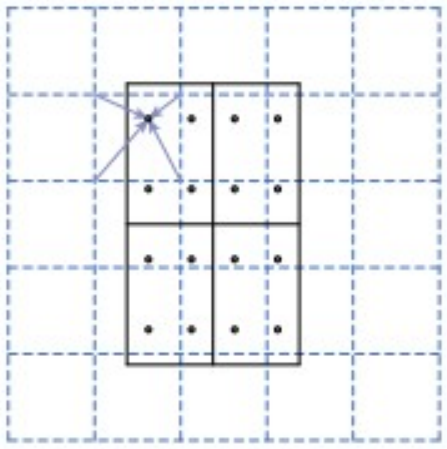


Figure 3. **RoIAlign:** The dashed grid represents a feature map, the solid lines an RoI (with 2×2 bins in this example), and the dots the 4 sampling points in each bin. RoIAlign computes the value of each sampling point by bilinear interpolation from the nearby grid points on the feature map. No quantization is performed on any coordinates involved in the RoI, its bins, or the sampling points.

RoI Align Layer in action. Source: Mask R-CNN, He et al., 2018

Instance Segmentation – Mask-RCNN Results

	backbone	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
MNC [10]	ResNet-101-C4	24.6	44.3	24.8	4.7	25.9	43.6
FCIS [26] +OHEM	ResNet-101-C5-dilated	29.2	49.5	-	7.1	31.3	50.0
FCIS+++ [26] +OHEM	ResNet-101-C5-dilated	33.6	54.5	-	-	-	-
Mask R-CNN	ResNet-101-C4	33.1	54.9	34.8	12.1	35.6	51.1
Mask R-CNN	ResNet-101-FPN	35.7	58.0	37.8	15.5	38.1	52.4
Mask R-CNN	ResNeXt-101-FPN	**37.1**	**60.0**	**39.4**	**16.9**	**39.9**	**53.5**

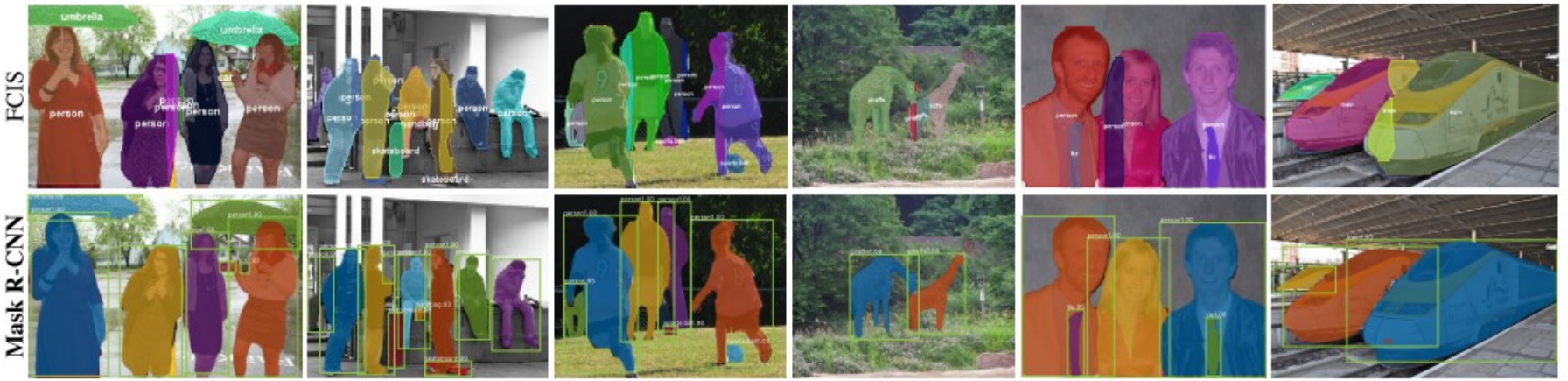


Figure 6. FCIS+++ [26] (top) *vs*. Mask R-CNN (bottom, ResNet-101-FPN). FCIS exhibits systematic artifacts on overlapping objects.

net-depth-features	AP	AP_{50}	AP_{75}
ResNet-50-C4	30.3	51.2	31.5
ResNet-101-C4	32.7	54.2	34.3
ResNet-50-FPN	33.6	55.2	35.3
ResNet-101-FPN	35.4	57.3	37.5
ResNeXt-101-FPN	**36.7**	**59.5**	**38.9**

(a) **Backbone Architecture**: Better backbones bring expected gains: deeper networks do better, FPN outperforms C4 features, and ResNeXt improves on ResNet.

	AP	AP_{50}	AP_{75}
softmax	24.8	44.1	25.1
sigmoid	**30.3**	**51.2**	**31.5**
	+5.5	*+7.1*	*+6.4*

(b) **Multinomial *vs*. Independent Masks** (ResNet-50-C4): *Decoupling* via per-class binary masks (sigmoid) gives large gains over multinomial masks (softmax).

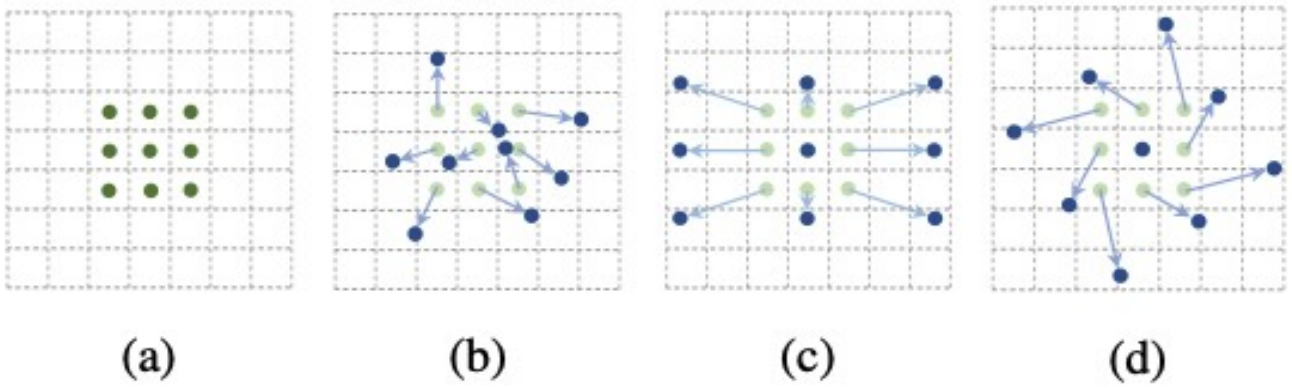
	align?	bilinear?	agg.	AP	AP_{50}	AP_{75}
RoIPool [12]			max	26.9	48.8	26.4
RoIWarp [10]		✓	max	27.2	49.2	27.1
		✓	ave	27.1	48.9	27.1
RoIAlign	✓	✓	max	**30.2**	**51.0**	**31.8**
	✓	✓	ave	**30.3**	**51.2**	**31.5**

(c) **RoIAlign** (ResNet-50-C4): Mask results with various RoI layers. Our RoIAlign layer improves AP by ~3 points and AP_{75} by ~5 points. Using proper alignment is the only factor that contributes to the large gap between RoI layers.

A note on Deformable Convolution

Although convolutions help with learning **spatially-local biases**, they suffer greatly from **signal-subsampling issues**:

- We are limited in processing patterns using **rectangular patterns**, whereas most natural patterns are not best suited to this template.
- We are limited in sampling points only from the discrete grid. Think of an 8x8 grid in which the object of interest is 2x2. We would not have a direct correspondent in a latent 2x2 grid which for said object due to downsampling.
  - In this case, ideally we should be able to **query fractional positions in the grid** for a 0.5x0.5 pixel



A note on Deformable Convolution

Deformable convolutions work similarly to RoI Pool / Alling operations:

- They can learn non-rectangular patterns by computing pixel offsets.
  - For a given filter, instead of applying it to its discrete neighbours, we compute a set of (k, dx, dy) offsets based on which new pixels will be selected.
  - Since the (dx, dy) values can be fractional, we can now query sub-pixels features: i.e. query at a pixel at a position such as (1, 0.5)
  - Ideally we would like to have such offsets for each spatial position in each filter - but this will blow-up memory, we're required to compute ($CHWk^2$) additional values at each stage.

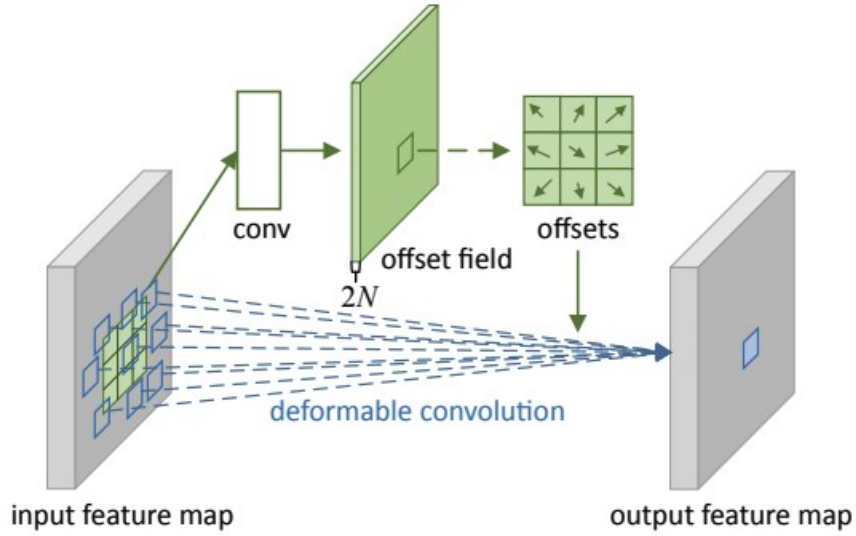


Figure 2: Illustration of 3×3 deformable convolution.

A note on Deformable Convolution

The deformation mechanism can be used to build more general mechanisms:

- We make a trade-off between modelling capacity and compute, by setting a constant spatial-offset (k, x, y) for each channel C
- We can retrieve an adaptive-pooling mechanism.
  - Average pooling is just a convolution with a uniform kernel with a value of 1/K, where K is the total filter size
  - Learning a scalar for each k pixel in the filter allows us to learn a sampling operation that can be a mixture between (**Average,Max,Min)** operations or something more general.
- This scaling mechanism can be extended to the normal deformable convolution as well
  - This can be a double-edged sword, since we might not use a kernel at it's full capacity. Scaling a pixel **k** in the kernel with a value of 0, renders the pixel useless.

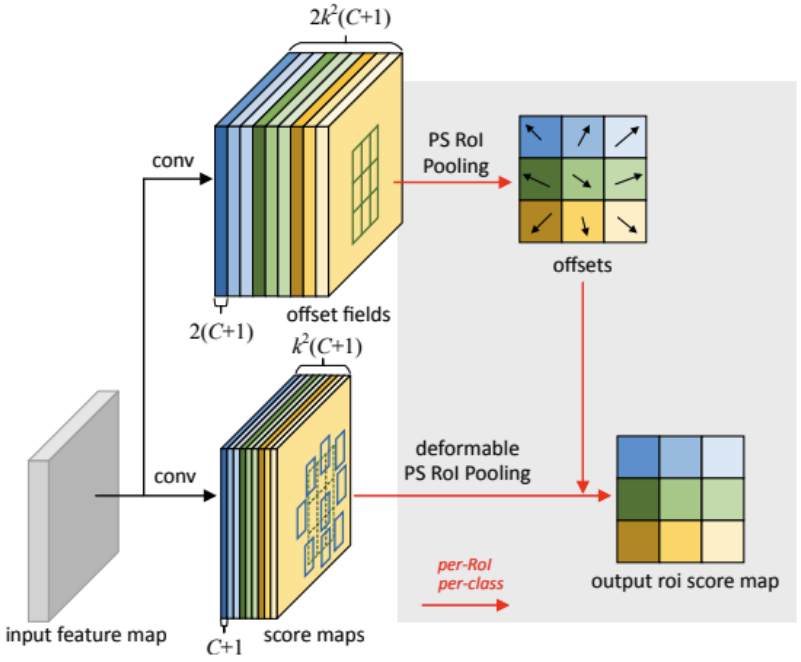


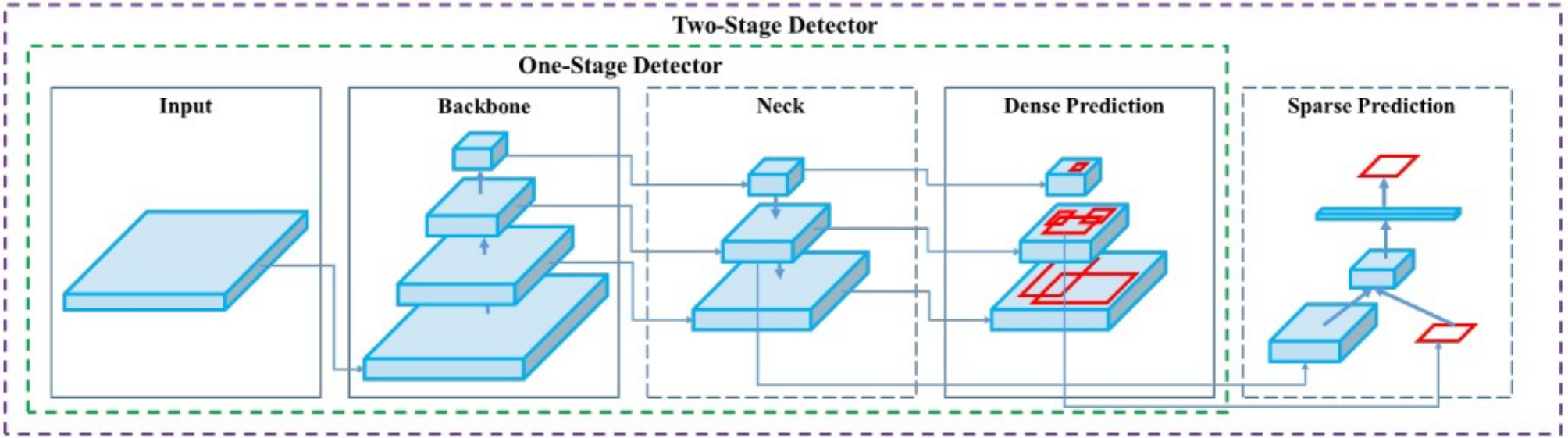
Figure 4: Illustration of 3×3 deformable PS RoI pooling.

Object Detection

Object Detection Approaches

- Two main approaches to Object Detection
  - Multi stage predictors - region proposals and region classification / bounding box regression happen in separate steps): e.g. Fast/Faster R-CNN, R-FCN
  - Single stage predictors - use a single forward pass through the architecture to perform both object classification and object bounding box regression at the same time: e.g. YOLO, SSD, RetinaNet

Object Detection Approaches



Input: { Image, Patches, Image Pyramid, … }

Backbone: { VGG16 [68], ResNet-50 [26], ResNeXt-101 [86], Darknet53 [63], … }

Neck: { FPN [44], PANet [49], Bi-FPN [77], … }

Head:

Dense Prediction: { RPN [64], YOLO [61, 62, 63], SSD [50], RetinaNet [45], FCOS [78], … }

Sparse Prediction: { Faster R-CNN [64], R-FCN [9], … }

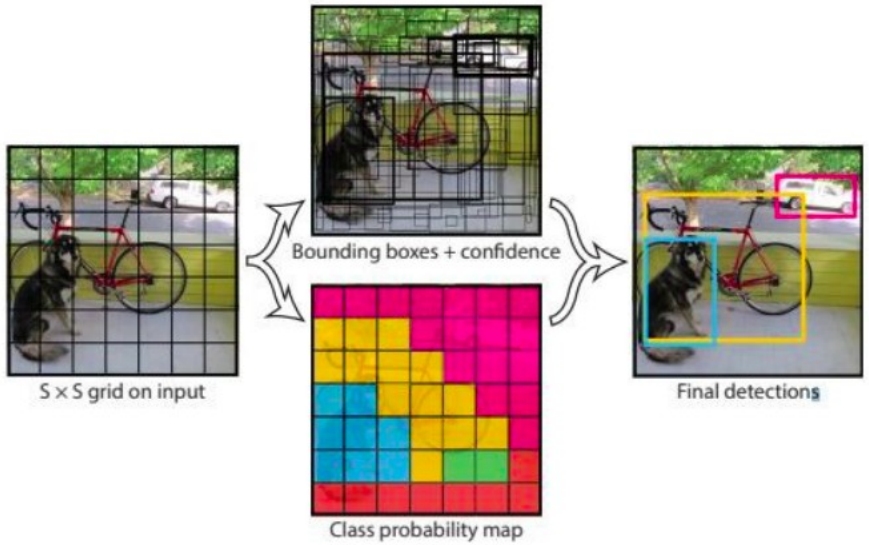
The “anatomy” of an object detector. Source YOLOv4, Bochkovskiy et al., 2020

A History of YOLOs - YOLO v1

- Processes frames in real time (45 fps)
- Trains on full image and has a **single** pipeline for detection **and** localization (as opposed to competitors such as R-CNN)
  - Has high detection accuracy
  - Has lower (than competitors at the time) localization accuracy

A History of YOLOs - YOLOv1 Algorithm

- YOLO algorithm works using the following three techniques:
  - Residual blocks
  - Bounding box regression
  - Intersection Over Union (IOU)
- Conceptual design diagram

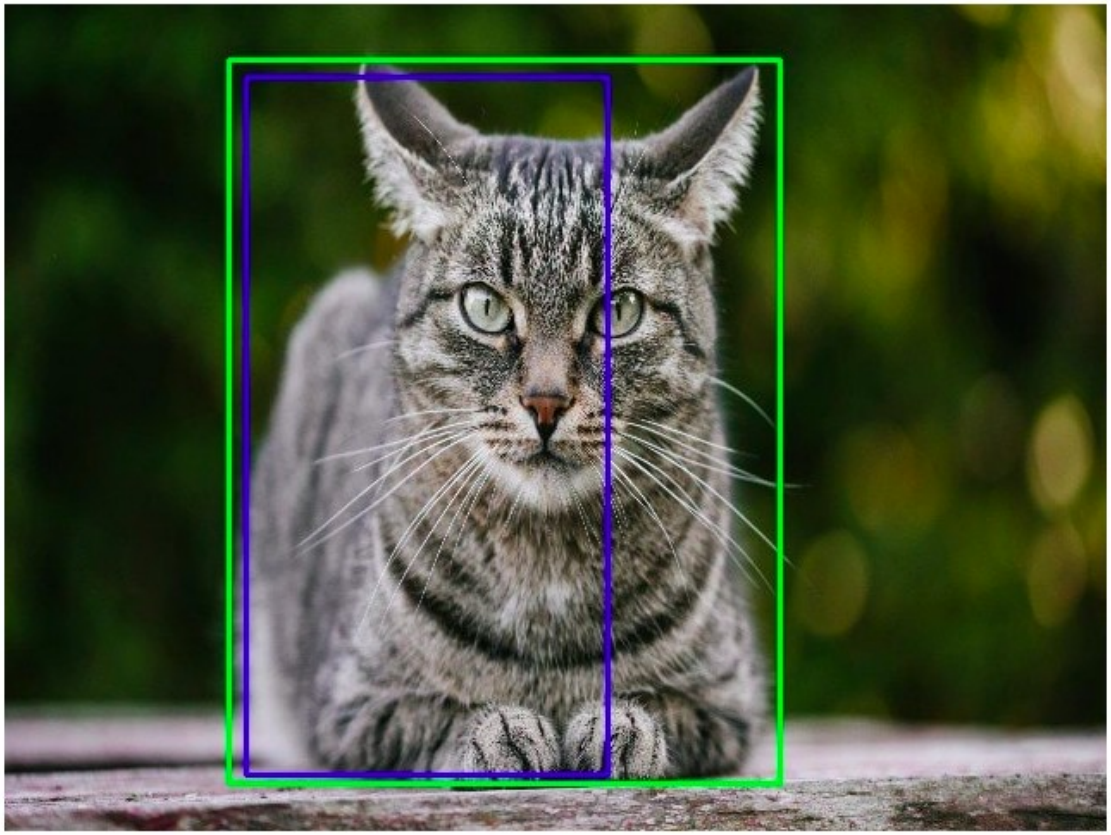


A History of YOLOs - YOLO v1 Algorithm

- Bounding box regression
  - 5 parameters per bounding box:
    - (x, y) - relative to the bounds of the grid cell)
    - width, height – relative to whole image
    - c – confidence = how confident the model is that the box contains an object and also how accurate it thinks the box is that it predicts
      - Confidence = $Pr(Object) * IOU^{truth}_{pred}$
- Each grid cell predicts B bounding boxes + class conditional probability
  - class conditional probability $p(class_i \mid Object)$ – conditioning is on grid cell containing an object
- => YOLO makes S x S x (B x 5 + C) predictions

A History of YOLOs - YOLO v1 Algorithm

- Intersection over Union
  - Metric used to force predicted output box to coincide with ground truth



A History of YOLOs - YOLO v1 Algorithm

- Loss Function:
  - Regression objective: optimize **directly** for detection of objects

$$\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

$$+ \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right]$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2$$

$$+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2$$

$$+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} \left(p_i(c) - \hat{p}_i(c) \right)^2$$

where $\mathbb{1}_i^{\text{obj}}$ denotes if object appears in cell i and $\mathbb{1}_{ij}^{\text{obj}}$ denotes that the jth bounding box predictor in cell i is "responsible" for that prediction.

A History of YOLOs - YOLO v1 Architecture

- YOLO architecture:
  - Inspired by GoogLe Net - 24 convolution layers + 2 Fully connected
  - Activation function : ReLU function (all layers except final), Linear Func (final layer)
  - Input image of 448x448x3
  - to avoid overfitting we use Dropout Layer and extensive data augmentation.

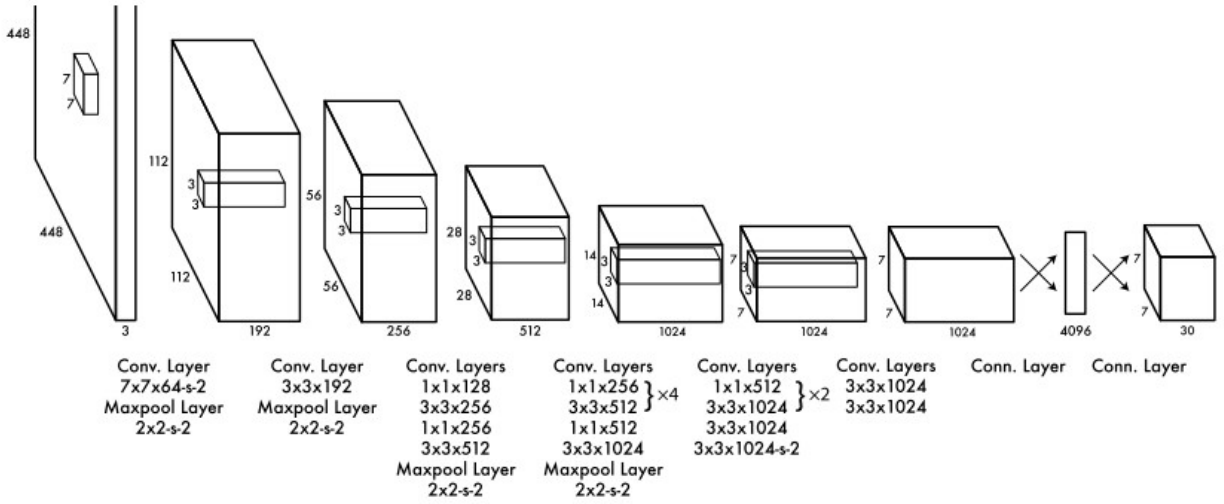


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

A History of YOLOs - YOLO v1 Algorithm

- YOLO training procedure:
  - **Pretrain first 20 conv layers** + avgpool + fc on ImageNet (for 1 week :-)), at image resolution of 224 x 224
  - Add 4 conv layers and 2 FC layers with random initialization when fine tuning for detection at 448 x 448 resolution

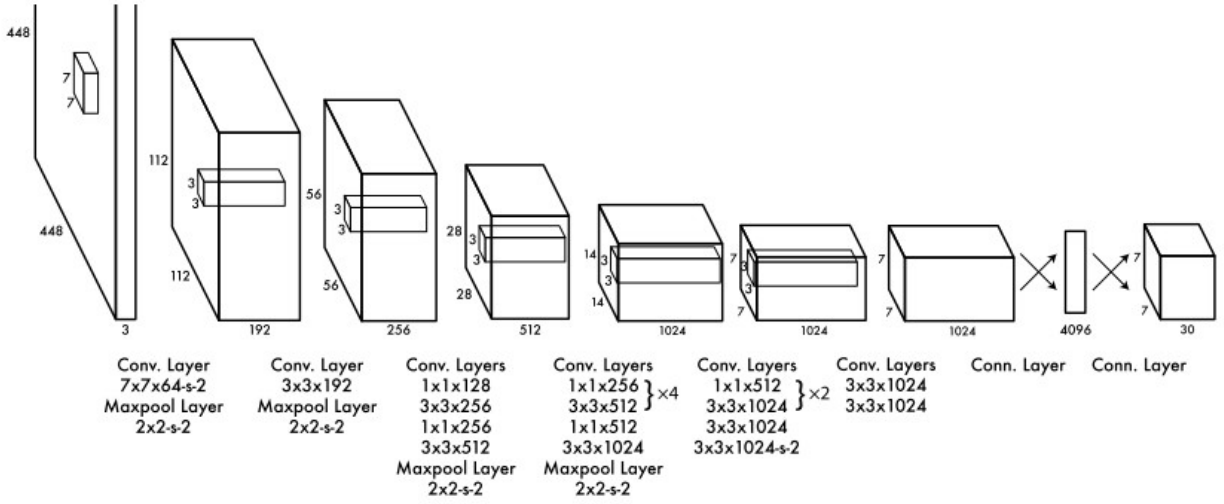


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

A History of YOLOs - YOLO v1 Limitations

- Original YOLO limitations:
  - Struggles with objects of small sizes that appear in groups (due to strong spatial constraints on bounding box predictions – each grid cell only predicts two boxes and can only have one class)
  - Struggles to generalize to objects in new or unusual aspect ratios – since bounding boxes are **learned** from data
  - Loss function treats errors the same in small bounding boxes versus large bounding boxes => incorrect localizations

A History of YOLOs - YOLOv2

- Differences to v1:
  - 23 Conv layers instead of 24 in YOLO v1
  - Activation function : **Leaky ReLU** function (all layers except final), Linear Func (final layer)
  - **Batch Normalization**: BN on all layers in YOLOv1 → 2% improvement in mAP. Helps regularize the model, allowing to remove Dropout layer.
  - Pretrain conv network directly on 448x448 resolution for ImageNet classification → 4% gain in mAP
  - Use the concept of **anchor boxes** – predefined list of *best match* box sizes (determined using k-means clustering on labeled train set). Predicted box is scaled w.r.t. defined anchor boxes
    - Predict **box center** w.r.t. *top left corner* of the grid cell (scaled by grid width and height)
    - Predict width and height of the box w.r.t. an anchor box
  - Multi-scale training – every 10th batch network trained on different input size (image resolution) → facilitate detections at different scales

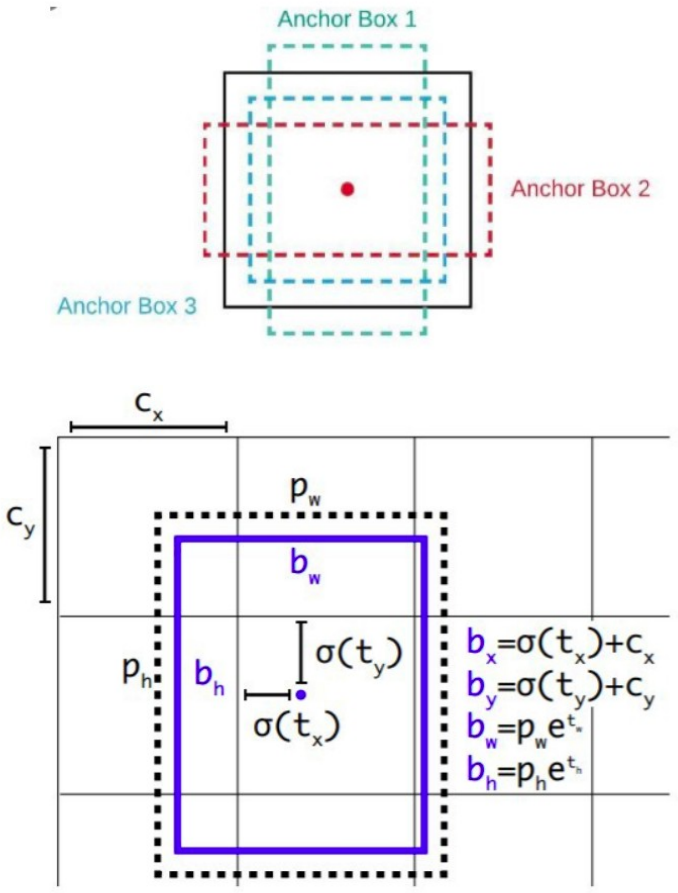


Figure 3: Bounding boxes with dimension priors and location prediction. We predict the width and height of the box as offsets from cluster centroids. We predict the center coordinates of the box relative to the location of filter application using a sigmoid function.

A History of YOLOs - YOLOv2

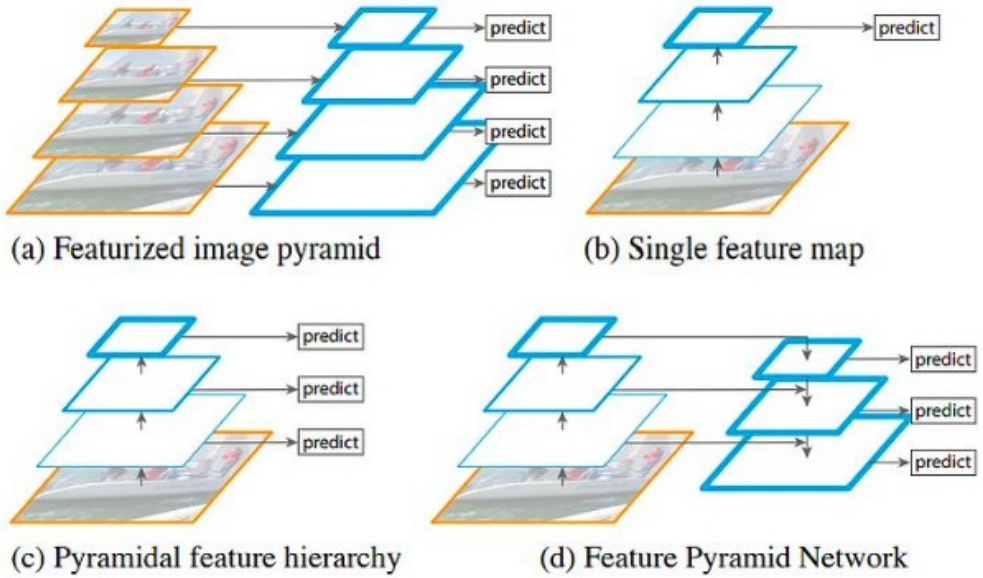
- Filtering out boxes of low quality
  - Filter out boxes with low confidence (as in YOLOv1)
  - Filter out boxes with high overlap (high IOU) – non maxima suppression – keep box with highest confidence scores

A History of YOLOs - YOLO v3

- Highlights of YOLOv3:
  - Improvement focus on accuracy (especially on small objects) and speed
  - 106 layer network, using 75 conv layers
    - Uses 23 residual layers at regular intervals to tackle *vanishing gradient* problem
  - Predictions **at different scales**
  - Darknet-53 used as feature extractor (part that can be pre-trained)

A History of YOLOs - YOLO v3

- Differences to v1 and v2:
  - Predict *objectness score* for each Bb using logistic regression
  - Width and height of box predicted as offsets from cluster centroids (anchors)
  - Center coordinates of Bbox predicted as relative to location of filter application using sigmoid function
  - Object class classifier uses **logistic function** instead of softmax (to help with multi-label classification), i.e. classification uses individual binary cross-entropy for each class label. Allows training on MS Open Images Dataset which has multiple labels per detection.
  - Prediction across scales – use of **Feature Pyramid Networks** – YOLOv3 uses 3 different scales

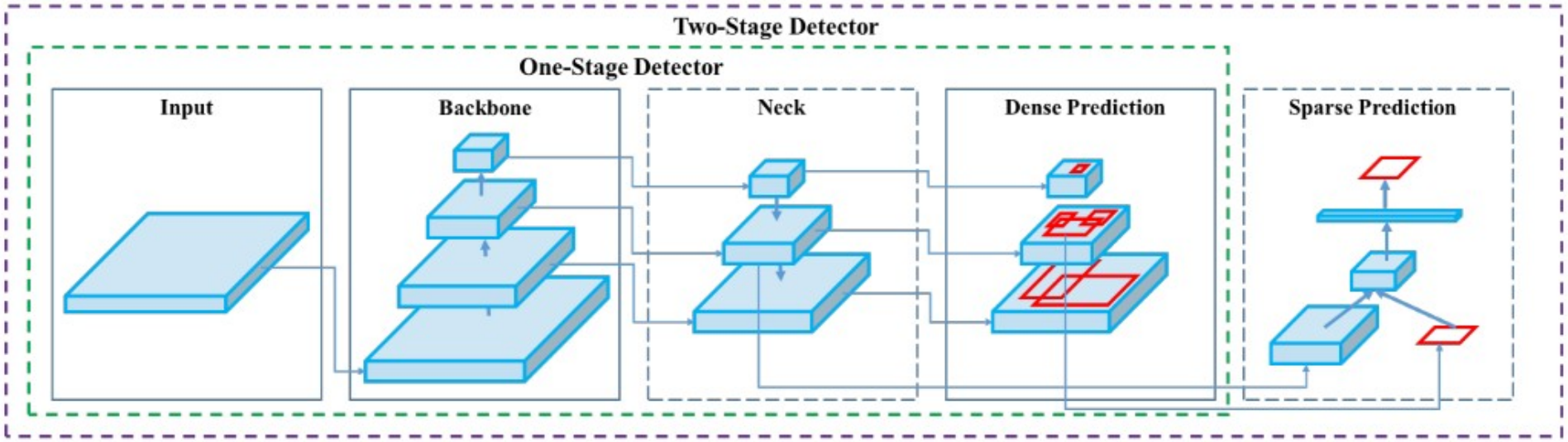


A History of YOLOs - YOLOv4 and YOLOv5

- Highlights of YOLOv4, YOLOv5:
  - Released within one month of each other – v5 is mostly a PyTorch implementation of v4 (developed using the Darknet framework)
  - Much higher detection speed – running at almost 140 fps
  - Improved performance on small object detection (primarily due to **mozaique data augmentation technique**)
  - Extensive test of **backbone network**
  - Use of **Path Aggregation Net** and **Spatial Pyramid Pooling (SPP)** blocks to aggregate features from several stages (scales) of the feature extractor network

A History of YOLOs - YOLO v4 and YOLOv5

- YOLOv4/v5 template architecture



Input: { Image, Patches, Image Pyramid, … }

Backbone: { VGG16 [68], ResNet-50 [26], ResNeXt-101 [86], Darknet53 [63], … }

Neck: { FPN [44], PANet [49], Bi-FPN [77], … }

Head:

Dense Prediction: { RPN [64], YOLO [61, 62, 63], SSD [50], RetinaNet [45], FCOS [78], … }

Sparse Prediction: { Faster R-CNN [64], R-FCN [9], … }

A History of YOLOs - YOLO v4 and YOLOv5

- YOLOv4 backbone selection
  - CSPResNext50 and CSPDarknet are both based on DenseNet
  - YOLOv4 chooses CSDarknet53 due to compromise in **receptive field size** (the higher the resolution, the higher the chances for small object detection) and **FPS**

Table 1: Parameters of neural networks for image classification.

Backbone model	Input network resolution	Receptive field size	Parameters	Average size of layer output (WxHxC)	BFLOPs (512x512 network resolution)	FPS (GPU RTX 2070)
CSPResNext50	512x512	425x425	20.6 M	**1058 K**	31 (15.5 FMA)	62
CSPDarknet53	512x512	725x725	**27.6 M**	950 K	52 (26.0 FMA)	**66**
EfficientNet-B3 (ours)	512x512	**1311x1311**	12.0 M	668 K	11 (5.5 FMA)	26

- YOLOv4 neck selection
  - **Path Aggregation Network** selected as multi-scale feature extractor
    - **Bottom up Path Augmentation** → propagate low-level features to enhance entire feature hierarchy
    - **Adaptive Feature Pooling** → for a RoI proposal aggregate features from all levels of the bottom up path
    - **Fully connected fusion** → exploit inductive bias in FC layer → for **detection** relations of features from *different* spatial positions matter

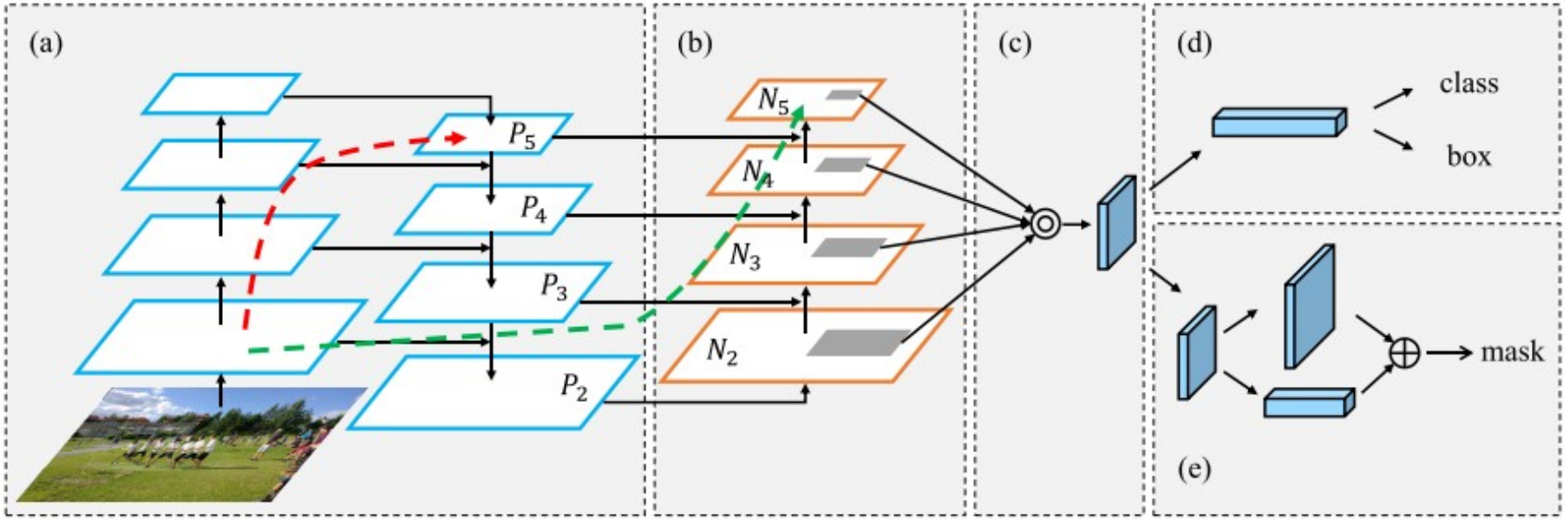
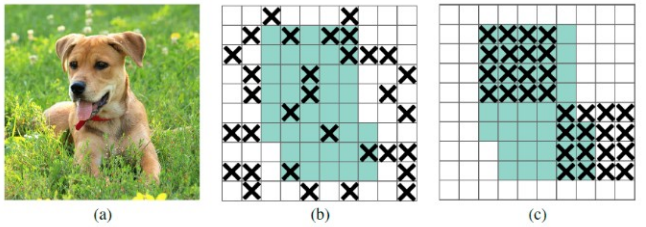


Figure 1. Illustration of our framework. (a) FPN backbone. (b) Bottom-up path augmentation. (c) Adaptive feature pooling. (d) Box branch. (e) Fully-connected fusion. Note that we omit channel dimension of feature maps in (a) and (b) for brevity.

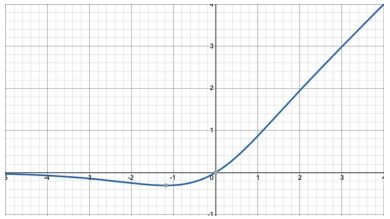
A History of YOLOs - YOLO v4 and YOLOv5

- YOLOv4 additional modifications (*Bag of Freebies* and *Bag of Specials*). Some highlights:
  - Data Augmentation: Mosaic Data Augmentation (works well for small object detection)
  - Regularization: DropBlock regularization, Class label smoothing
  - Activation Functions: MISH activation
  - Loss: CioU (complete IoU loss) – addresses problem of non-overlapping bounding boxes; proportional to overlapping area, distance and aspect ration



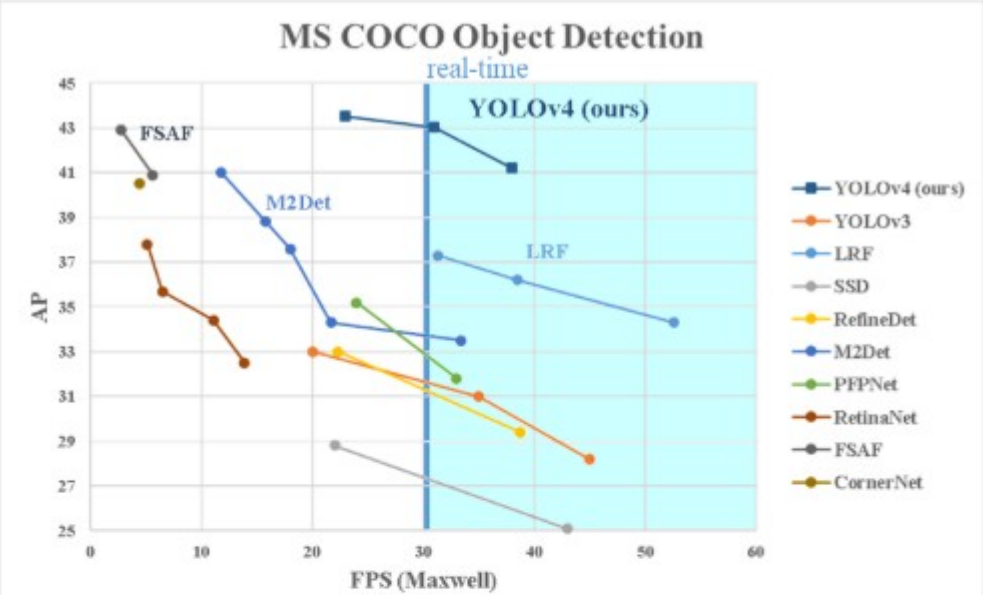


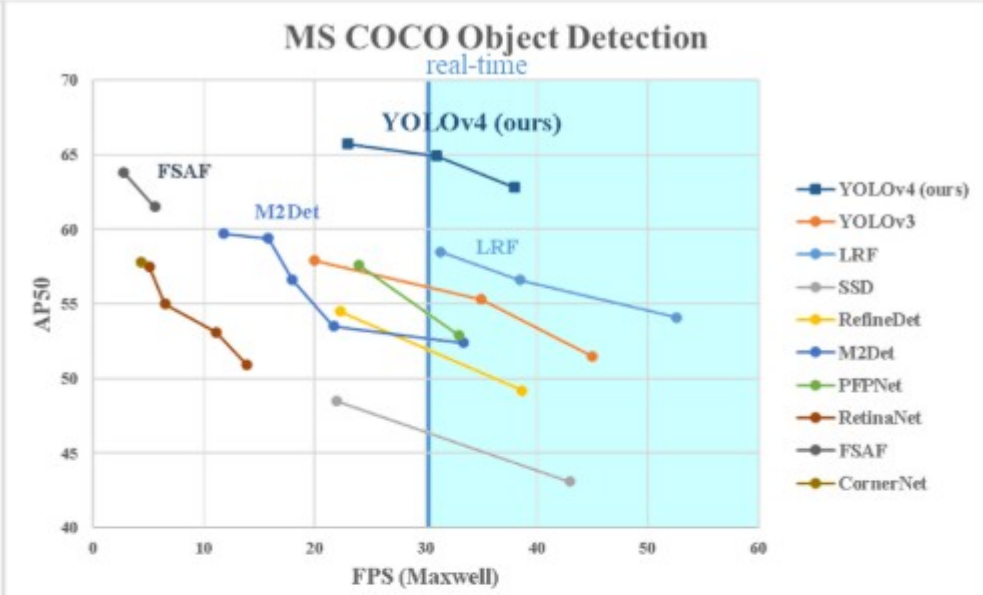
(a) Input Image, (b) Dropout Randomly at Feature Maps, (c) DropBlock at Feature Maps

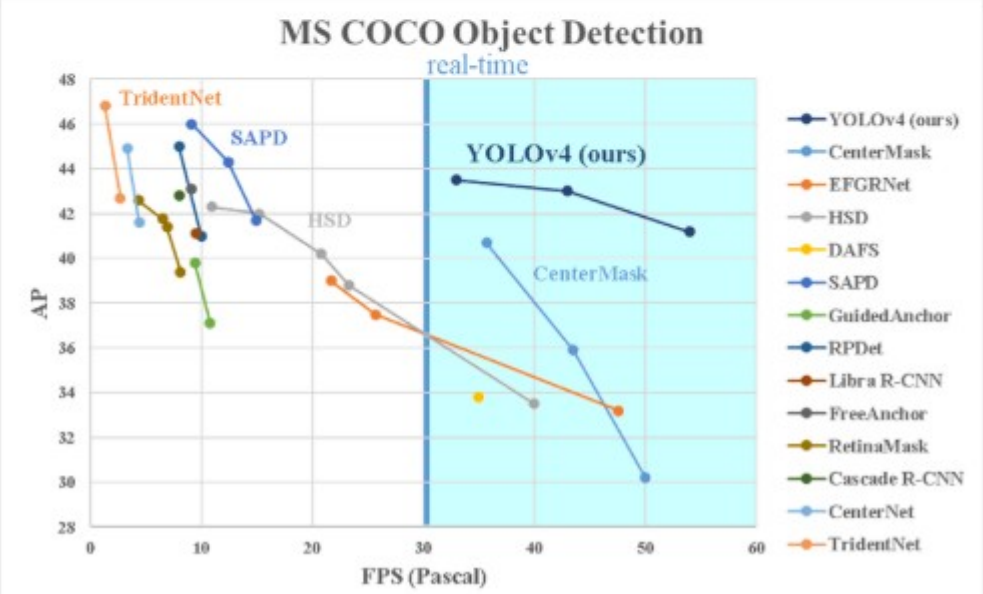


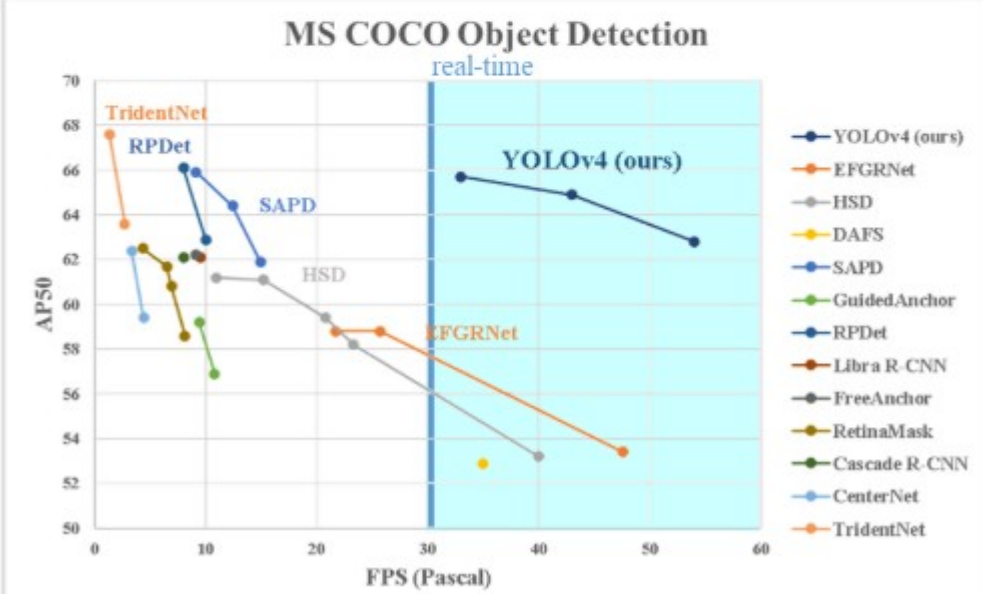
$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v.$$

A History of YOLOs - YOLO v4 and YOLOv5 results









Convolutions for Time Series Modeling

Convolutions for Time Series Modeling

- Why Convolutional Architectures for *sequence data?*
  - More efficient to train than Recurrent Neural Net architectures (see next lecture)
  - Have the comparable (if not better performance) as RNN architectures, especially on multi-variate numeric time series (e.g. sensor data streams)
  - Variable length inputs can be handled by sliding the convolution kernel along the sequence input

Convolutions for Time Series - TCN

- **Temporal Convolutional Network (TCN)**
  - Introduced in article: “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling” BAI et al., 2018
  - Highlights
    - **1D conv kernels** are used to make an output sequence have *the same length* as the input sequence
    - Use of **causal convolutions**
      - There can no *leakage* from the past
      - The convolution kernel that ouputs $\mathbf{y_j}$ **only looks** at inputs $\mathbf{x_i}$ where **i <= j**
    - Use of **dilated convolutions** to extend **receptive field** and increase “historical context” when predicting the output sequence
    - Uses a **residual block** instead of single conv layers → increases max depth of network → increases “historical context” modeling capabilities

Convolutions for Time Series - TCN

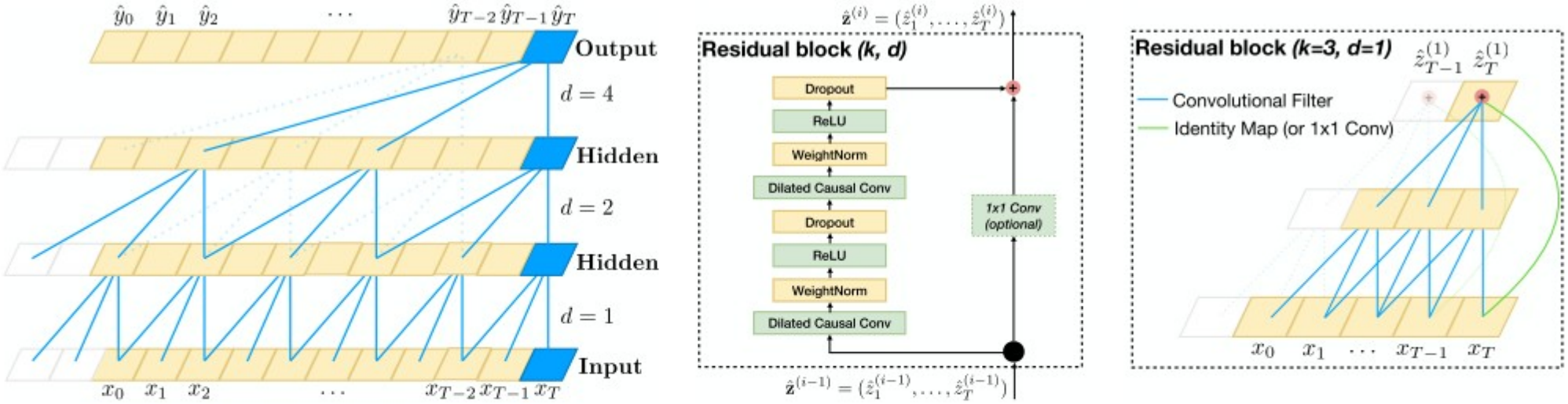


Figure 1. Architectural elements in a TCN. (a) A dilated causal convolution with dilation factors $d = 1, 2, 4$ and filter size $k = 3$. The receptive field is able to cover all values from the input sequence. (b) TCN residual block. An 1x1 convolution is added when residual input and output have different dimensions. (c) An example of residual connection in a TCN. The blue lines are filters in the residual function, and the green lines are identity mappings.

Source: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, Bai et al., 2018

Convolutions for Time Series - TCN

- **Temporal Convolutional Networks – strengths and limitations**
  - **Strengths:**
    - **Parallelism** – due to application of conv filter => input sequence can be processed as a whole, not sequentially
    - **Flexible receptive field size**: stack more causal conv layers, increase dilation factor, increase filter size
    - **Stable gradients**: use of residual blocks, backprop easier than for RNNs
  - **Limitations**
    - In test/evaluation mode, the network needs the whole input sequence in memory, instead of just the hidden state vectors (as in the case of RNNs – see next lecture)
    - May require parameter change for **transfer learning**
      - If transfering to a domain where the *needed* "historical context" is greater (i.e. larger filter size or larger dilation factor are needed), TCNs may perform poorly

Convolutions for Time Series - TCN

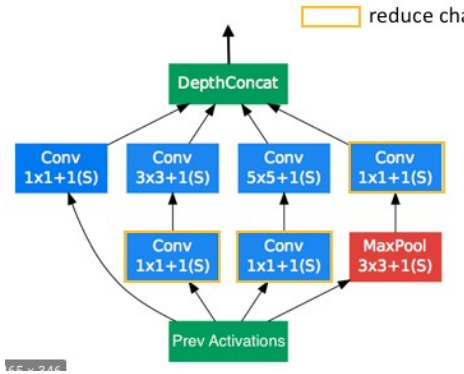
Table 1. Evaluation of TCNs and recurrent architectures on synthetic stress tests, polyphonic music modeling, character-level language modeling, and word-level language modeling. The generic TCN architecture outperforms canonical recurrent networks across a comprehensive suite of tasks and datasets. Current state-of-the-art results are listed in the supplement. ^{h} means that higher is better. $^{\ell}$ means that lower is better.

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	**TCN**
Seq. MNIST (accuracy^{h})	70K	87.2	96.2	21.5	**99.0**
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	**97.2**
Adding problem T=600 (loss$^{\ell}$)	70K	0.164	**5.3e-5**	0.177	**5.8e-5**
Copy memory T=1000 (loss)	16K	0.0204	0.0197	0.0202	**3.5e-5**
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	**8.10**
Music Nottingham (loss)	1M	3.29	3.46	4.05	**3.07**
Word-level PTB (perplexity$^{\ell}$)	13M	**78.93**	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	**45.19**
Word-level LAMBADA (perplexity)	-	4186	-	14725	**1279**
Char-level PTB (bpc$^{\ell}$)	3M	1.36	1.37	1.48	**1.31**
Char-level text8 (bpc)	5M	1.50	1.53	1.69	**1.45**

Convolutions for Time Series - InceptionTime

- **InceptionTime**
  - Introduced in article: “InceptionTime: Finding AlexNet for Time Series Classification”, Fawaz et al
  - Highlights
    - Focuses on **time series classification**
    - Uses the **multiple perspectives** principle introduced in GoogleNet (Inception Cell) and applies it to time series → extract relevant features from both long and short time series
    - Uses **ensembling** (in a bagging manner – initialize the same network architecture 5 times and average the results) to reduce variance in classification performance

Convolutions for Time Series - InceptionTime



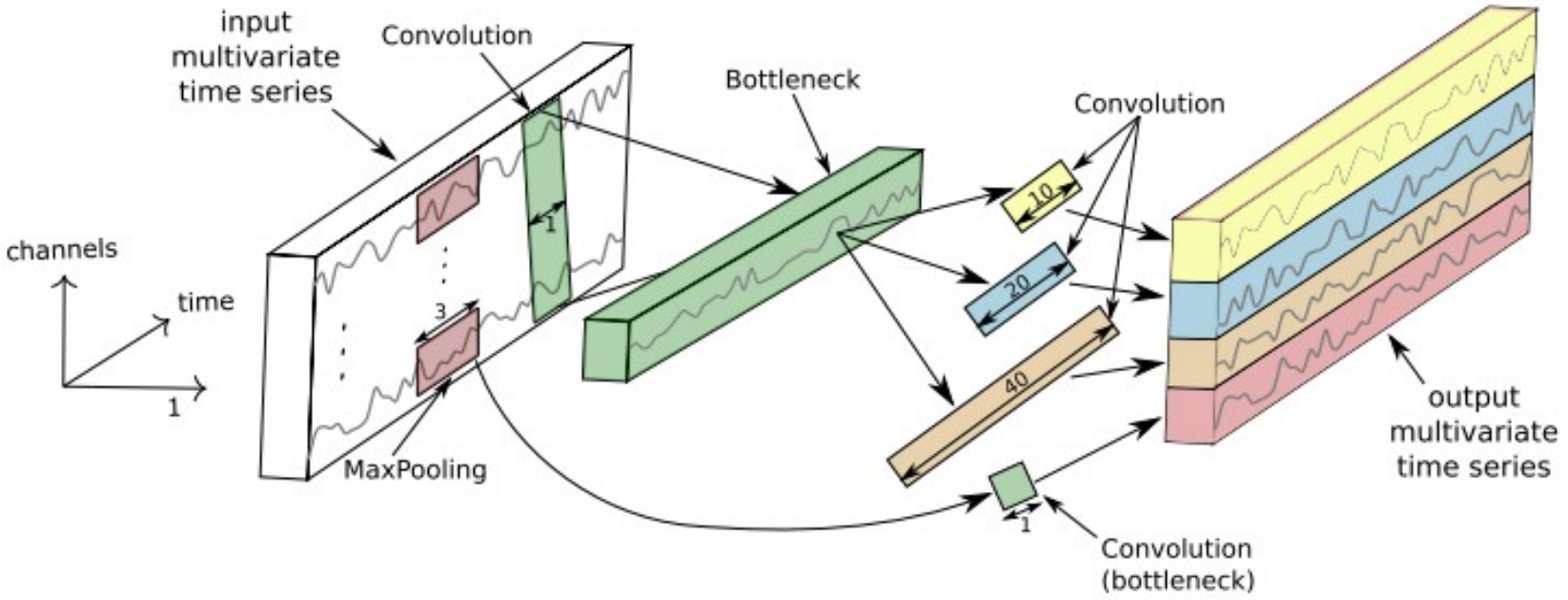


Fig. 1: Inside our Inception module for time series classification

Convolutions for Time Series - InceptionTime

- **InceptionTime Network**
  - 2 residual blocks, each composed of 3 inception modules
  - Global average pooling applied before a FC layer that gives the final classification dimension
  - Ensemble model to reduce variance in classification accuracy of single models

$$\hat{y}_{i,c} = \frac{1}{n}\sum_{j=1}^{n}\sigma_c(x_i, \theta_j) \quad | \quad \forall c \in [1, C]$$

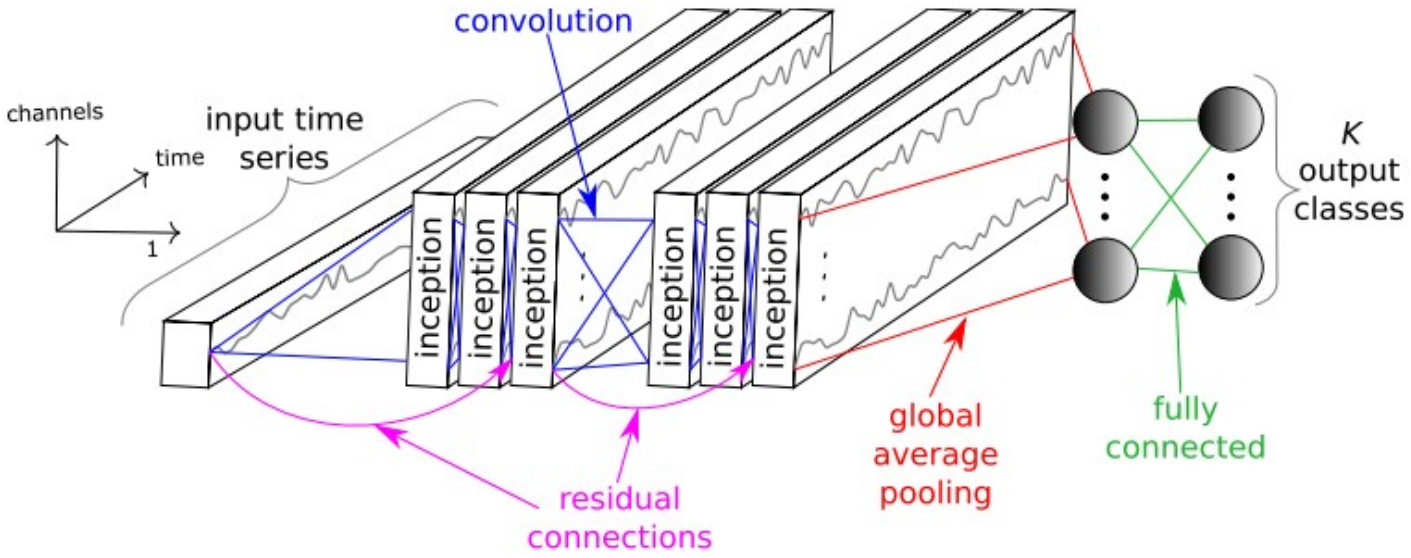


Fig. 2: Our Inception network for time series classification

The end :-)